

from Beginner to Master

DevOps를 위한 **Docker** 가상화

Section 7. Docker Security

```
book:
  def setData(self, title, price, author):
    self.title = title
    self.price = price
    self.author = author

var fs = require('fs');
fs.readFile('./ONE.txt', '* 1 *',
  function (err, data) {
    console.log(data); // 3
  });
ApplicationDelegate > {
```

```
public static void main(String[] args)

<servlet>
  <servlet-name>BoardController</servlet-name>
  <servlet-class>com.joneconsulting.controller.BoardCont
  <init-param>
    <param-name>user_name</param-name>
    <param-value>henneth Lee</param-value>
  </init-param>
</servlet>
```

Winterface

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

수정하기

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명



멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명

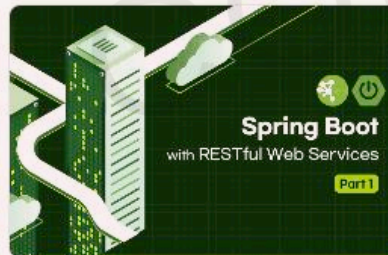


Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

이 되었던 때가 있었습니
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구
사람입니다. 개발자로서, 강
지만, 그래도, 남들보다 조

ecture, Blockchain,

- Section 0: Introduction to Cloud Native Technologies
- Section 1: Docker Essentials - Container
- Section 2: Docker Essentials - Image
- Section 3: Docker Network and Storage
- Section 4: Building and Managing Containerized Application
- Section 5: Container Orchestration
- Section 6: Continuous Integration and Continuous Deployment
- **Section 7: Docker Security**
- Section 8: Logging and Monitoring
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

Section 7.

Docker Security

- Docker와 Cloud Native Application 보안
- Docker 보안
- Docker에서의 정보 관리

Cloud Native Application 보안

- 컨테이너, 서버리스, 마이크로서비스 등 다양한 클라우드 기반 기술을 활용하는 애플리케이션의 전체 생명 주기에 걸쳐 보안을 적용
- 멀티 클라우드 환경에서의 데이터 보호, 동적 보안 정책 적용, 연속적인 보안

istio를 활용하여 서비스 간 TLS 통신 강제 적용

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: default
  namespace: my-namespace
spec:
  host: "*.my-namespace.svc.cluster.local"
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
```

Docker와 Cloud Native Application 보안

Docker 보안 vs Cloud Native Application 보안

	Docker 보안	Cloud Native Application 보안
범위	컨테이너 수준에서의 보안 조치에 집중	보다 광범위한 클라우드 환경과 여러 서비스를 통합하여 보안 강화
접근 방식	- 이미지와 컨테이너의 취약점 관리에 초점 - 컨테이너 런타임 시 보안 정보 사용	동적 환경에서의 정책 기반 보안과 통합된 로깅/모니터링 전략 사용
기술적 요소	- 이미지 보안 - 컨테이너 격리	Cloud Native Application을 구성하는 서비스들의 보안 기술 사용

Docker Security

- Docker Daemon Security
 - | 컨테이너 실행을 위한 Docker Engine에 대한 보안
- Container Image Security
 - | 컨테이너 이미지가 안전하게 생성 되었는지 검증 → 이미지에 대한 보안 취약성 문제점 확인
- Container Isolation
 - | 컨테이너의 실행이 다른 컨테이너나 Host에 영향을 주지 않는 독립적인 실행 환경 → Isolation
- Runtime Security
 - | 컨테이너 실행 동안 지속적인 모니터링과 관리

Docker Daemon Security

- Docker daemon(dockerd)는 Root 권한을 가지고 실행되기 때문에 주의 깊은 보안

| Limit Root Access
관리 필요

| TLS (Transport Layer Security) 활성화 (SSL, Https)

```
{
  "tls": true,
  "tlscacert": "/path/to/ca.pem",
  "tlscert": "/path/to/server-cert.pem",
  "tlskey": "/path/to/server-key.pem",
  "hosts": ["tcp://0.0.0.0:2376", "unix:///var/run/docker.sock"]
}
```

| API Access 제한

Container Image Security

- 보안 취약점 문제가 포함되어 있지 않는 컨테이너 이미지 사용
 - | 컨테이너 이미지는 취약점에 민감함 레이어와 sw로 구성 → 검증된 기본 이미지(Base Image) 사용
 - | 보안 취약점 점검 → Clair, Trivy, Docker Security Scanning
 - | 이미지 크기 최소화 → alpine 이미지 등
- 컨테이너의 기반이 되는 도커 이미지의 보안을 확보하고 취약점이 없도록 하기 위해 이미지에 대한 취약점 검사
 - | Clair, Trivy 또는 도커 자체의 스캔 기능을 사용하여 정기적으로 도커 이미지를 취약점 검사

Container Image Security

■ Docker Scout

- | 이미지를 분석하여 Software Bill of Materials (SBOM, SW 자재 명세)라고 하는 구성 요소 인벤토리를 컴파일
- | Docker Desktop에서는 기본 제공, Linux Docker에서는 설치 필요 (<https://docs.docker.com/scout/install>)
- | Docker Desktop, Docker Hub, Docker CLI, Docker Scout 대시보드와 상호 작용가능한 독립한 서비스 플랫폼

■ Docker Image 내 보안 위협 탐색

```
$ docker scout quickview edowon0623/docker:latest
```

```
C:\work\git>docker scout quickview edowon0623/docker:latest
i New version 1.7.0 available (installed version is 1.2.0) at https://github.com/docker/scout-cli
v Image stored for indexing
v Indexed 617 packages

Target          edowon0623/docker: latest
digest          0fb6edbcfae2
Base image      centos:8
Updated base image centos:7

CVSS score      9C      154H      247M      66L      30?
Severity rating Low (L)
Medium (M)
High (H)
Critical (C)

CVSS score      0C      17H      119M      40L
Severity rating Low (L)
Medium (M)
High (H)
Critical (C)

CVSS score      1C      14H      35M      13L
Severity rating Low (L)
Medium (M)
High (H)
Critical (C)

CVSS score      +1      -3      -84      -27
```

CVSS score	Severity rating
0.1 – 3.9	Low (L)
4.0 – 6.9	Medium (M)
7.0 – 8.9	High (H)
9.0 – 10.0	Critical (C)

```
$ docker scout recommendations edowon0623/first-service
```

Container Image Security

■ Trivy

- | Aqua Security에서 개발한 Trivy는 보안을 개발 프로세스에 원활하게 통합하기 위해 개발
- | Trivy는 컨테이너의 취약점 및 구성 문제를 쉽게 검색할 수 있도록 설계된 보안 스캐너
- | 취약성 탐지의 정확성이 높으며, 데이터베이스 설치를 위한 사전 요구 사항이 없음
- | OS 패키지(Alpine, RHEL, CentOS 등) 및 언어별 패키지(Bundler, Composer, npm, Yarn 등)를 검색
- | 포괄적인 취약점 감지(OS, 미들웨어, 애플리케이션 라이브러리)
- | 설치가 간단하고 CI/CD 파이프라인에 쉽게 통합

Container Image Security (실습)

■ Trivy 설치

| <https://aquasecurity.github.io/trivy/v0.18.3/installation/>

```
$ docker pull aquasec/trivy:0.18.3
```

| Scan a Docker image

```
$ docker run --rm -v /home/ec2-user/.cache:/root/.cache/ aquasec/trivy:0.18.3 edowon0623/first-service
```

```
edowon0623/first-service (debian 10.8)
=====
Total: 198 (UNKNOWN: 5, LOW: 14, MEDIUM: 81, HIGH: 74, CRITICAL: 24)

+-----+-----+-----+-----+-----+-----+
| LIBRARY | VULNERABILITY ID | SEVERITY | INSTALLED VERSION | FIXED VERSION | TITLE |
+-----+-----+-----+-----+-----+-----+
| apt | CVE-2011-3374 | LOW | 1.8.2.2 | | It was found that apt-key in apt, |
| | | | | | all versions, do not correctly... |
| | | | | | -->avd.aquasec.com/nvd/cve-2011-3374 |
+-----+-----+-----+-----+-----+-----+
| bash | CVE-2019-18276 | HIGH | 5.0-4 | | bash: when effective UID is not |
| | | | | | equal to its real UID the... |
| | | | | | -->avd.aquasec.com/nvd/cve-2019-18276 |
+-----+-----+-----+-----+-----+-----+
```

Container Isolation

- 컨테이너와 호스트는 기본적으로 격리 → 무단 액세스를 방지 및 위험을 최소화

이러한 위해 격리 강화 → 프로세스, 네트워킹, 파일 시스템을 격리

Control Groups (cgroups) → 리소스를 보호하기 위해 CPU, Memory, I/O 리소스 제한

```
$ docker run -it --cpus=".5" --memory="512MB" <IMAGE_NAME>
```

- 컨테이너 내의 사용자 ID를 호스트의 다른 사용자 ID에 매핑하여 권한 상승 공격의

위험 감소 → Container Root 사용자를 Non-root 사용자(보안이 강화 된)와 매핑

```
{  
  "usersns-remap": "default"  
}
```

```
[ec2-user@ip-172-31-5-111 ~]$ sudo dockerd --usersns-remap=default  
INFO[2024-04-21T21:55:31.839844844Z] Starting up  
WARN[2024-04-21T21:55:31.840001198Z] Running experimental build  
INFO[2024-04-21T21:55:31.840188604Z] User namespaces: ID ranges will be mapped to subuid/subgid ranges of: dockremap  
Can't create ID mappings: no subuid ranges found for user "dockremap"
```

Container Isolation

- 도커의 기본 네트워크 구성은 도커 네트워크 내에서 자유롭게 사용 가능
- 컨테이너 간의 트래픽을 제한하기 위해 네트워크 세분화 및 방화벽 규칙 필요

| 사용자 정의 브리지 네트워크를 생성하여 컨테이너 통신을 격리

| 격리 된 Docker Network 생성

```
$ docker network create --driver bridge isolated_network
```

| 격리 된 Docker Network에 컨테이너 연결

```
$ docker run --network=isolated_network <IMAGE_NAME>
```

Runtime Security

- 실행 중인 컨테이너 보안 강화 및 지속적인 모니터링

| Security Profiles → AppArmor, SELinux, seccomp profiles 사용

```
$ docker run --security-opt seccomp=/path/to/seccomp-profile.json <IMAGE_NAME>
```

- 컨테이너를 최소 권한으로 실행 → 취약점을 이용한 공격의 위험 감소

| root 사용자는 운영체제 전반적인 권한을 가짐 → 보안 상 고려해야 할 요소가 많은 계정

| Linux Capabilities → root 슈퍼유저의 권한을 세분화하여 부분적으로 권한을 허용하는 역할

```
chown, dac_override, fowner, fsetid, kill, setgid, setuid, setpcap, net_bind_service, net_raw, sys_chroot, mknod, audit_write, setfcap
```

```
$ docker run --cap-drop=ALL --cap-add=SETGID --cap-add=SETUID --cap-add=CHOWN \
--cap-add=DAC_OVERRIDE --cap-add=FOWNER -it first-image:0.1 bash
```

Runtime Security

■ Dockerfile에서의 권한 조정

- | Dockerfile 로 이미지를 빌드할 때는 높은 권한으로 실행되기 때문에 제한 필요 없음
- | 컨테이너 이미지를 이용하여 컨테이너를 실행할 때 제한 필요

```
FROM ubuntu:20.04
RUN apt-get update && apt-get install -y curl wget
# Add your application setup here
# ...
CMD ["bash"]
```

```
$ docker build -t myimage .
```

```
$ docker run --cap-drop=ALL --cap-add=SETGID --cap-add=SETUID --cap-add=CHOWN \
--cap-add=DAC_OVERRIDE --cap-add=FOWNER -it <IMAGE_NAME>
```


Runtime Security

- 컨테이너 권한 관리
- 컨테이너를 최소 권한으로 실행 → 취약점을 이용한 공격의 위험 감소
- 가능한 경우 컨테이너를 root로 실행하지 않도록
 - | Dockerfile에서 USER 지시어를 사용하여 컨테이너를 root가 아닌 사용자로 실행하도록 지정

```
FROM nginx:latest  
RUN adduser -D nonrootuser  
USER nonrootuser
```

Runtime Security (실습)

- Capabilities 제어

```
$ docker run -it first-image:0.1 bash
```

```
# apt-get update
```

```
$ docker run -it --cap-drop=ALL first-image:0.1 bash
```

```
# apt-get update
```

```
$ docker run --cap-drop=ALL --cap-add=SETGID --cap-add=SETUID --cap-add=CHOWN \  
--cap-add=DAC_OVERRIDE --cap-add=FOWNER -it first-image:0.1 bash
```

```
# apt-get update
```

Secrets Management

- Secret은 보안이 필요한 민감한 데이터를 클러스터 단위에서 안전하게 관리하는
용도로 사용
Docker Swarm Mode에서 생성된 Service에서 사용 가능
- Docker Secrets 또는 외부 비밀 관리 서비스를 사용하여 민감한 데이터를 안전하게 저장하지 않도록
- HashiCorp Vault
주의
Create a Docker secret (Docker Swarm 설정 필요)

```
$ docker swarm init # Initialize a swarm if not already done  
  
$ echo "MySecretData" | docker secret create my_secret -  
  
$ echo qwer1234 | docker secret create my_db_password -
```

Secrets Management

- Docker Swarm의 Secret에 있는 데이터는 항상 암호화
- Secret에 포함된 데이터는 해당 Secret이 삽입된 Service와, 이에 속하여 구동 중인 Task의 Container에서만 접근 가능
- Docker Swarm의 Secret은 파일 형태로 컨테이너에 마운트

| Secret 목록 조회

```
$ docker secret ls
```

```
$ docker secret inspect my_secret
```

| 서비스 실행 시 Secret 사용

```
$ docker service create --name my-redis --secret my_secret redis:alpine
```

```
redis container# redis-cli set my_secret_key "$(cat /run/secrets/my_secret)"
```

Secrets Management

■ Docker Service 적용

| Service 생성

```
$ docker service create \
    --name my-db \
    --replicas 1 \
    --secret source=my_db_password,target=mariadb_root_password \
    -e MARIADB_ROOT_PASSWORD_FILE="/run/secrets/mariadb_root_password" \
    -e MARIADB_DATABASE="mydb" \
    mariadb:latest
```

| Mysql 접속 시 Secret 암호 사용

```
$ docker exec -it [container_id] bash
```

```
docker_container # mariadb -h127.0.0.1 -uroot -p
```

로깅 및 모니터링 Logging and Monitoring

- 컨테이너 활동을 모니터링
- 로그를 유지 관리하여 보안 사고를 감지하고 문제를 해결하는 데 도움
 - | Docker Logging Driver를 구성하여 로그를 안전하고 외부 시스템에 중앙 집중화하여 분석
 - | Syslog를 이용하여 컨테이너 실행

```
$ docker run --log-driver syslog --log-opt syslog-address=udp://192.168.0.42:514 <IMAGE_NAME>
```

정기 업데이트 Regular Updates

- 도커 엔진, 도커 이미지 및 호스트 운영 체제를 정기적으로 업데이트하여 최신
도커와 호스트 시스템을 업데이트하고 업데이트된 이미지에서 컨테이너를 재배포
보안 패치 적용